Réf : .../..... **AU : 2025–2026**

Université de Sousse

École Nationale d'Ingénieurs de Sousse

Projet Module Big Data

IoT Patient Monitoring System

Conception et déploiement d'un pipeline Big Data temps réel

Réalisé par : Iyed Ben Salem & Mouadh Chagra

Filière : Génie des Technologies Embarquées (GTE2)

Encadrant : T. Abdellatif

Année : 2025–2026

Date de rendu : 27 Avril 2026

Structure du Projet

Le projet est organisé en un répertoire auto-contenu. Les composants sont séparés par responsabilité : simulation, traitement, stockage et initialisation.

```
iot-patient-monitoring/                    # Répertoire racine du projet
|- docker-compose.yml                      # Orchestration des services (Kafka, MongoDB, Spark, Mongo Express)
|- Data/                                   # Répertoire des données
|  |- sensors.csv                          # 10 000 relevés IoT synthétiques (auto-générés)
|  `- generate_data.py                     # Génère sensors.csv avec Faker + NumPy (seed=42)
|- producer/                               # Couche simulateur IoT
|  `- gateway.py                           # Lit le CSV, classifie les relevés, publie sur Kafka
|- consumer/                               # Couche traitement et alertes
|  |- spark_consumer.py                    # PySpark Streaming :  Kafka -> classification -> MongoDB + alertes
|  `- alert_consumer.py                    # Écoute les topics d'alerte, affiche en console
|- setup/                                  # Initialisation unique
|  |- init_mongo.py                        # Crée les collections patients, devices et les index
|  `- create_topics.py                     # Crée 7 topics Kafka via KafkaAdminClient
|- inject_critical.py                      # Force un relevé CRITIQUE pour la démo live
`- venv/                                   # Environnement virtuel Python (exclu du git)
```

Configuration Docker Compose

Toute l'infrastructure est définie dans un unique `docker-compose.yml`. Les services communiquent via le réseau bridge interne de Docker ; seuls les ports nécessaires sont exposés à l'hôte.

Service	Image	Port	Rôle
zookeeper	`cp-zookeeper:7.4.0`	2181	Coordinateur du broker Kafka
kafka	`cp-kafka:7.4.0`	9092	Broker de messages, routage des topics
mongodb	`mongo:6.0`	27017	Stockage persistant des documents
mongo-express	`mongo-express:latest`	8083	Interface web pour MongoDB
spark-master	`apache/spark:3.5.0`	8082, 7077	Coordinateur du cluster Spark
spark-worker	`apache/spark:3.5.0`	—	Exécution des jobs de streaming

Listing 1: Fichier `docker-compose.yml` complet

```yaml
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment: { ZOOKEEPER_CLIENT_PORT: 2181, ZOOKEEPER_TICK_TIME: 2000 }
    ports: ["2181:2181"]

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  mongodb:
    image: mongo:6.0
    ports: ["27017:27017"]
    environment: { MONGO_INITDB_ROOT_USERNAME: root, MONGO_INITDB_ROOT_PASSWORD: root }
    volumes: [mongo_data:/data/db]

  mongo-express:
    image: mongo-express:latest
    depends_on: [mongodb]
    ports: ["8083:8081"]
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: root
      ME_CONFIG_MONGODB_ADMINPASSWORD: root
      ME_CONFIG_MONGODB_URL: mongodb://root:root@mongodb:27017/
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin123

  spark-master:
    image: apache/spark:3.5.0
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master
    ports: ["8082:8080", "7077:7077"]

  spark-worker:
    image: apache/spark:3.5.0
    depends_on: [spark-master]
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker spark://spark-master:7077

volumes:
```

```
  mongo_data:
```

Description du Jeu de Données

Source et justification

Le jeu de données est entièrement **synthétique**, généré par `Data/generate_data.py` avec `Faker`, `NumPy` et `random` (seed = 42 pour la reproductibilité). Aucune donnée réelle de patient n'est utilisée, garantissant la conformité RGPD. Cette approche assure une conformité exacte du schéma et permet l'injection à la demande de valeurs extrêmes pour la démonstration.

Référence : Jeu de données synthétique auto-généré. Script : `Data/generate_data.py`.

Propriété	Valeur
Total des enregistrements	10 000 relevés de capteurs
Patients	10 patients uniques (P-0001 à P-0010)
Types de dispositifs	5 (FC, Température, SpO2, Pression artérielle, Fréq. respiratoire)
Métriques distinctes	5
Plage temporelle	2024-03-01 au 2024-06-23 (un relevé toutes les 30 s)
Format de fichier	CSV, UTF-8, ≈600 Ko

Topics Kafka et justification des partitions

Nom du topic	Parts.	Objectif et justification
`patient.heartrate`	3	Fréquence cardiaque – 3 partitions correspondent aux cœurs du worker Spark
`patient.temperature`	3	Température – volume élevé, consommateurs parallèles
`patient.oxygenlevel`	3	SpO2 – métrique critique, faible latence requise
`patient.bloodpressure`	3	Pression artérielle – stratégie de parallélisme uniforme
`patient.respiratoryrate`	3	Fréquence respiratoire – cohérence avec les autres métriques
`patient.alerts.critical`	1	Alertes critiques – partition unique garantissant l'ordre
`patient.alerts.warning`	1	Alertes d'avertissement – faible volume, traitement séquentiel strict

Seuils d'alerte

Métrique	Normal	WARNING	CRITICAL	Unité
Fréq. cardiaque	60–100	<50 ou >120	<40 ou >150	bpm
Température	36.1–37.2	>38.0	>39.5 ou <35	°C
OxygenLevel	95–100	<93	<90	SpO2%
Pression artérielle	80–120	>130	>180 ou <70	mmHg
Fréq. respiratoire	12–20	<10 ou >24	<8 ou >30	br/min

Guide d'utilisation – Du clone à la démo live

Prérequis

Ubuntu 22.04/24.04 • Docker Engine 24+ avec Compose v2 • Python 3.10+ avec `python3-venv` • Git

Étape 1 – Cloner et préparer l'environnement Python

```
git clone https://github.com/<username>/iot-patient-monitoring.git
cd iot-patient-monitoring
python3 -m venv venv && source venv/bin/activate
pip install kafka-python pymongo pandas numpy faker pyspark
```

Étape 2 – Démarrer tous les services Docker

```
docker compose up -d
docker ps     # 6 conteneurs attendus : zookeeper, kafka, mongodb,
              #                         mongo-express, spark-master, spark-worker
```

Attendre ≈30 secondes pour que Kafka et MongoDB s'initialisent complètement avant de continuer.

Étape 3 – Générer le jeu de données et initialiser

```
python3 Data/generate_data.py     # cree Data/sensors.csv (10 000 lignes)
python3 setup/init_mongo.py       # cree les collections : patients, devices + index
python3 setup/create_topics.py    # cree 7 topics Kafka
```

Étape 4 – Lancer le pipeline (3 terminaux)

Ouvrir trois terminaux, activer le venv dans chacun (`source venv/bin/activate`) :

#	Terminal	Commande à exécuter
A	Moniteur d'alertes	`python3 consumer/alert_consumer.py`
B	Consumer Spark	`python3 consumer/spark_consumer.py`
C	Simulateur IoT	`python3 producer/gateway.py`

Le terminal A affichera les alertes CRITICAL et WARNING en temps réel tandis que le terminal C diffuse les données dans le pipeline.

Étape 5 – Déclencher une alerte CRITIQUE (démo live)

```
python3 inject_critical.py
# Envoie FC = 180 bpm pour le patient P-0001
# => alerte CRITIQUE immediate visible dans le terminal A
```

Étape 6 – Vérifier les données et les interfaces web

Interface	URL	Identifiants
Mongo Express (UI DB)	`http://localhost:8083`	`admin / admin123`
Spark Master UI	`http://localhost:8082`	Aucun requis
MongoDB (direct)	`localhost:27017`	`root / root`

Dans Mongo Express : `patient_monitoring > sensor_readings` affiche tous les documents avec leur champ `alertLevel`. Dans l'UI Spark : **Status: ALIVE**, 1 Worker et 1 Running Application confirment que le job de streaming est actif.

Étape 7 – Arrêter le système

```
docker compose down       # arrete les conteneurs (donnees preservees dans le volume)
docker compose down -v    # remise a zero complete incluant le volume MongoDB
```

Pipeline complet : `sensors.csv` $\xrightarrow{\text{gateway.py}}$ **Topics Kafka** $\xrightarrow{\text{spark_consumer.py}}$ **MongoDB + topics d'alertes** $\xrightarrow{\text{alert_consumer.py}}$ **Sortie console**

— Fin du rapport — Iyed Ben Salem & Mouadh Chagra | GTE2 | ENISO 2025–2026 —